Animation Using Key-Frames & Python

Ball Animation

Anargyros Sarafopoulos

National Centre for Computer Animation
Bournemouth University

workshop notes

Outline

1 Preparing To Animate The Ball

2 Timing and Spacing & References

Moving The Ball Along The (Vertical) Y-axis

Gravity & Friction

- We want the ball to accelerate as it is falling under the force of gravity.
- We also want the ball to gradually lose energy by friction. This should occur as it is colliding with the floor and air particles.
- Eventually, depending on the size and material properties of the ball and friction with the ground and air particles, the ball should come to a halt.

Look at: "Bouncing Ball on-line Reference"

Moving The Ball Along The (Vertical) Y-axis

Gravity & Friction

- We want the ball to accelerate as it is falling under the force of gravity.
- We also want the ball to gradually lose energy by friction. This should occur as it is colliding with the floor and air particles.
- Eventually, depending on the size and material properties of the ball and friction with the ground and air particles, the ball should come to a halt.

Look at: "Bouncing Ball on-line Reference"

Moving The Ball Along The (Vertical) Y-axis

Gravity & Friction

- We want the ball to accelerate as it is falling under the force of gravity.
- We also want the ball to gradually lose energy by friction. This should occur as it is colliding with the floor and air particles.
- Eventually, depending on the size and material properties of the ball and friction with the ground and air particles, the ball should come to a halt.

Look at: "Bouncing Ball on-line Reference"

Friction With Air & Ground

The Ball Timing & Spacing With Friction

- We need to have some way of establishing, how fast the ball looses energy and how fast it stops.
- So, let us consider a simple model, where the ball looses the same amount of energy every time it hits the ground. Let, say it looses 20% of its energy at every impact.
- Let's again consider the ball loses a constant amount of energy by friction with air particles after each bounce or while falling. Say, this costs 10% of its energy after each bounce to max height and after falling to the ground.

Friction With Air & Ground

The Ball Timing & Spacing With Friction

- We need to have some way of establishing, how fast the ball looses energy and how fast it stops.
- So, let us consider a simple model, where the ball looses the same amount of energy every time it hits the ground. Let, say it looses 20% of its energy at every impact.
- Let's again consider the ball loses a constant amount of energy by friction with air particles after each bounce or while falling. Say, this costs 10% of its energy after each bounce to max height and after falling to the ground.

Friction With Air & Ground

The Ball Timing & Spacing With Friction

- We need to have some way of establishing, how fast the ball looses energy and how fast it stops.
- So, let us consider a simple model, where the ball looses the same amount of energy every time it hits the ground. Let, say it looses 20% of its energy at every impact.
- Let's again consider the ball loses a constant amount of energy by friction with air particles after each bounce or while falling. Say, this costs 10% of its energy after each bounce to max height and after falling to the ground.

Calculating the Ball Timing and Spacing

Ball Timing & Spacing Using Friction

- Using the above ideas we define two constants, friction with the ground $fr_{ground} = 0.2$ and friction with the air $fr_{air} = 0.1$. These affect the ball's timing and spacing.
- Let's for simplicity consider that fr_{ground} affects only the height of the ball's bounce (scale of the bounce or spacing) and fr_{air} affects how long it takes for the ball to hit the ground after each bounce (the timing of the bounce).
- Let's consider ground to be at zero and the ball starts falling from a height of twelve (12) units. We start at frame 1 and the timing of the first impact is at frame 12.

Calculating the Ball Timing and Spacing

Ball Timing & Spacing Using Friction

- Using the above ideas we define two constants, friction with the ground $fr_{ground} = 0.2$ and friction with the air $fr_{air} = 0.1$. These affect the ball's timing and spacing.
- Let's for simplicity consider that fr_{ground} affects only the height of the ball's bounce (scale of the bounce or spacing) and fr_{air} affects how long it takes for the ball to hit the ground after each bounce (the timing of the bounce).
- Let's consider ground to be at zero and the ball starts falling from a height of twelve (12) units. We start at frame 1 and the timing of the first impact is at frame 12.

Timing & Spacing Calculations

Timing & Spacing of Each Bounce

- We can now use Python to help us with the calculation of the ball's timing and spacing. Especially, calculation of the height (**spacing**) and duration of the ball's trajectory after each bounce (**timing**). We define the following variables.

```
1  # friction applied when ball hits the ground
2  fr_ground = 0.2
3  # Y-axis position which identifies a resting ball
4  rest_y = 0.0009 # it can't be zero ... !
5  # starting position (height) of the ball
6  pos  = 12.0
7  # time it takes to come in contact with the ground when
8  time_interval = 11 # falling from a height of 12.0 units
9  # empty list, to store ball maximum heights  after each bounce
10 pos_list = [] # stores maximum Y-coordinates after each bounce
```

Spacing Calculations

Height of Each Bounce

- We can now loop until the ball stops bouncing, while storing the corresponding maximum height values in the position list.

```
1  fr_ground = 0.2 # friction with the ground
2  rest_y = 0.0009 # resting ball
3  pos = 12.0      # starting position
4  time_interval = 11 # to reach ground from 12 units high
5  pos_list = []      # list of heights after each bounce
6  # loop to calculate maximum height positions
7  while pos >= rest_y:
8      pos *= (1.0-fr_ground) # friction applied
9      if pos >= rest_y:
10         pos_list.append(pos) # append position to list
```

Timing Calculations

Timing of the First Bounce

- We now want to calculate the times we set key-frames. We set keys when the ball is at its maximum and minimum positions along the Y-axis.
- We initialise relevant variables as shown and store the time of the key set at the first frame of the sequence.
- We increment time by 11 frames so the ball at the next key frame (given by “curr_time + time_interval”) is at its min Y-coordinate position on the ground.

```
1 fr_air     = 0.1 # % reduction in frames it takes to fall
2 time_list = []  # list to hold times of set key-frames
3 curr_time = 1   # current frame and time
4
5 time_list.append(1) # set key at frame 1
6 curr_time += time_interval # update time
```

Timing Calculations

Timing of the First Bounce

- We now want to calculate the times we set key-frames. We set keys when the ball is at its maximum and minimum positions along the Y-axis.
- We initialise relevant variables as shown and store the time of the key set at the first frame of the sequence.
- We increment time by 11 frames so the ball at the next key frame (given by "curr_time + time_interval") is at its min Y-coordinate position on the ground.

```
1 fr_air     = 0.1 # % reduction in frames it takes to fall
2 time_list = []  # list to hold times of set key-frames
3 curr_time = 1   # current frame and time
4
5 time_list.append(1) # set key at frame 1
6 curr_time += time_interval # update time
```

Timing Calculations

Timing of the First Bounce

- We now want to calculate the times we set key-frames. We set keys when the ball is at its maximum and minimum positions along the Y-axis.
- We initialise relevant variables as shown and store the time of the key set at the first frame of the sequence.
- We increment time by 11 frames so the ball at the next key frame (given by “curr_time + time_interval”) is at its min Y-coordinate position on the ground.

```
1 fr_air     = 0.1 # % reduction in frames it takes to fall
2 time_list = []  # list to hold times of set key-frames
3 curr_time = 1   # current frame and time
4
5 time_list.append(1) # set key at frame 1
6 curr_time += time_interval # update time
```

Timing Calculations List

Timing of Each Bounce

- We can now loop through all the calculated max height positions, while storing the times of corresponding key-frames between each bounce.

```
# loop through the position-list (we calculated at slide 7)
# in order to set the timing (the keys) of bounce events
for i in range(0, len(pos_list)):
    # ball at min position and now rising
    curr_time = round(curr_time) # keys at integer frames
    time_list.append(curr_time)  # set key at ''rest_y''
    time_interval *= (1.0-fr_air)  # decrease time interval
    curr_time += time_interval     # update current time
    # ball at max position and now falling
    curr_time = round(curr_time)
    time_list.append(curr_time)  # set key at ''pos_list[i]''
    # up and down movent for each bounce is the same
    # time_interval *= (1.0-fr_air)
    curr_time += time_interval
```

Table of Keys & Heights

Calculated Times & Positions

- Now, we can inspect the values calculated. The first and third row of the table hold frame times and the second and fourth the corresponding Y-coordinates (heights) of the ball.

1	12	22	31	39	46	52	58
12	0.001	9.600	0.001	7.680	0.001	6.144	0.001
63	68	72	76	79	82	85	88
4.915	0.001	3.932	0.001	3.146	0.001	2.517	0.001

Translation Along The X-axis

Ball Moving Forward

- Once we are happy with the up-down (Y-axis movement) we can start working on the forward movement of the ball.
- This is simpler than the Y-axis translations, as the movement along the horizontal axis is almost linear.
- So we can simply set only two key-frames, on at the start (say 0 X-axis) position and one at the end position.

Translation Along The X-axis

Ball Moving Forward

- Once we are happy with the up-down (Y-axis movement) we can start working on the forward movement of the ball.
- This is simpler than the Y-axis translations, as the movement along the horizontal axis is almost linear.
- So we can simply set only two key-frames, on at the start (say 0 X-axis) position and one at the end position.

Translation Along The X-axis

Ball Moving Forward

- Once we are happy with the up-down (Y-axis movement) we can start working on the forward movement of the ball.
- This is simpler than the Y-axis translations, as the movement along the horizontal axis is almost linear.
- So we can simply set only two key-frames, on at the start (say 0 X-axis) position and one at the end position.

Forward Movement

Setting Translation Keys Along the X-axis

- We set keys at the “start_frame” and “start_pos” and “last_frame” and “end_pos” (assuming these variables are initialised) as follows:

```
# add key-frames
import sys
import pymel.core as pm # import PyMel
# import scripts from current workspace
sys.path.append(pm.workspace(expandName='scripts'))
obj_name = 'ball_anim'
ball = pm.PyNode(obj_name) # ball PyNode object
cmds.currentTime(start_frame)
ball.translateX.set(start_pos)
cmds.setKeyframe(ball, attribute='translateX')
cmds.currentTime(last_frame)
ball.translateX.set(end_pos)
cmds.setKeyframe(ball, attribute='translateX')
```

Rotation Along The Z-axis

Rotating Forward

- Once we are happy with the translation of the ball along the Y and X-axis, we can start thinking of rotations.
- This is also simple, we will keep the rotational movement along the Z-axis also linear.
- So we can simply set only two key-frames, one at the start, say at 0 angle and one at the end angle.

Rotation Along The Z-axis

Rotating Forward

- Once we are happy with the translation of the ball along the Y and X-axis, we can start thinking of rotations.
- This is also simple, we will keep the rotational movement along the Z-axis also linear.
- So we can simply set only two key-frames, one at the start, say at 0 angle and one at the end angle.

Rotation Along The Z-axis

Rotating Forward

- Once we are happy with the translation of the ball along the Y and X-axis, we can start thinking of rotations.
- This is also simple, we will keep the rotational movement along the Z-axis also linear.
- So we can simply set only two key-frames, one at the start, say at 0 angle and one at the end angle.

Rotational Movement

Setting Rotation Keys Along the Z-axis

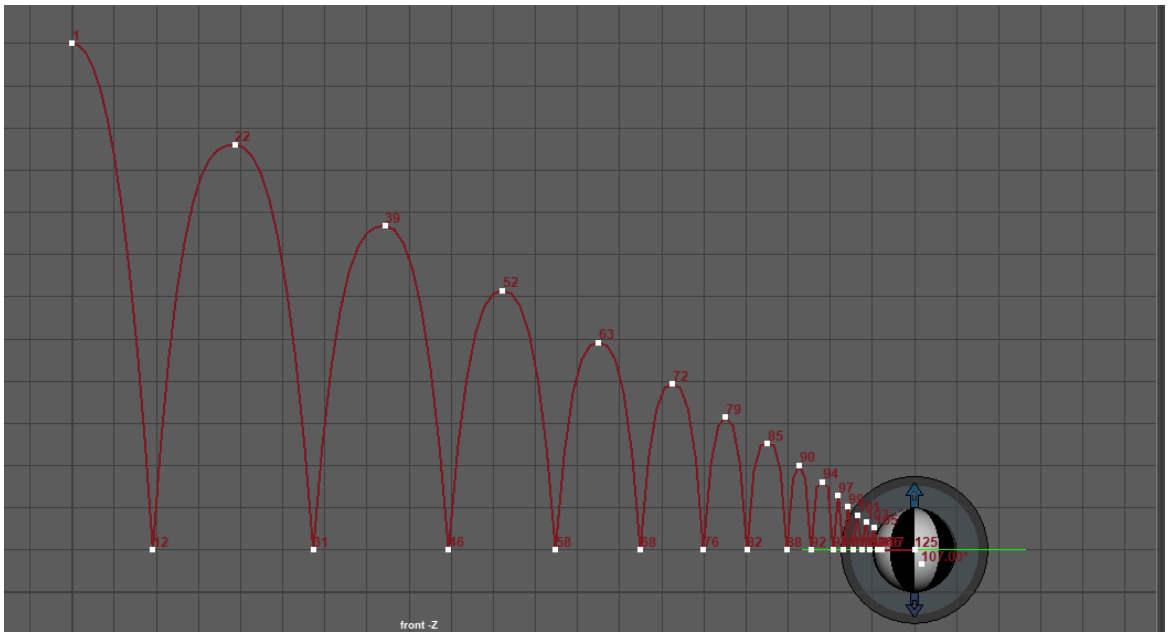
- We set keys at the “start_frame” and “start_rot” and “last_frame” and “end_rot” (assuming these variables are initialised) as follows:

```
# add key-frames
import sys
import pymel.core as pm # import PyMel
# import scripts from current workspace
sys.path.append(pm.workspace(expandName='scripts'))
obj_name = 'ball_anim'
ball = pm.PyNode(obj_name) # ball PyNode object
pm.currentTime(start_frame)
ball.rotateZ.set(start_rot)
pm.setKeyframe(ball, attribute='rotateZ')
# add at maximum one more second (25 frames) of movement
pm.currentTime(last_frame)
ball.rotateZ.set(end_rot)
pm.setKeyframe(ball, attribute='rotateZ')
```

Combined Movement

Movement Along X/Y-axis & Rotation

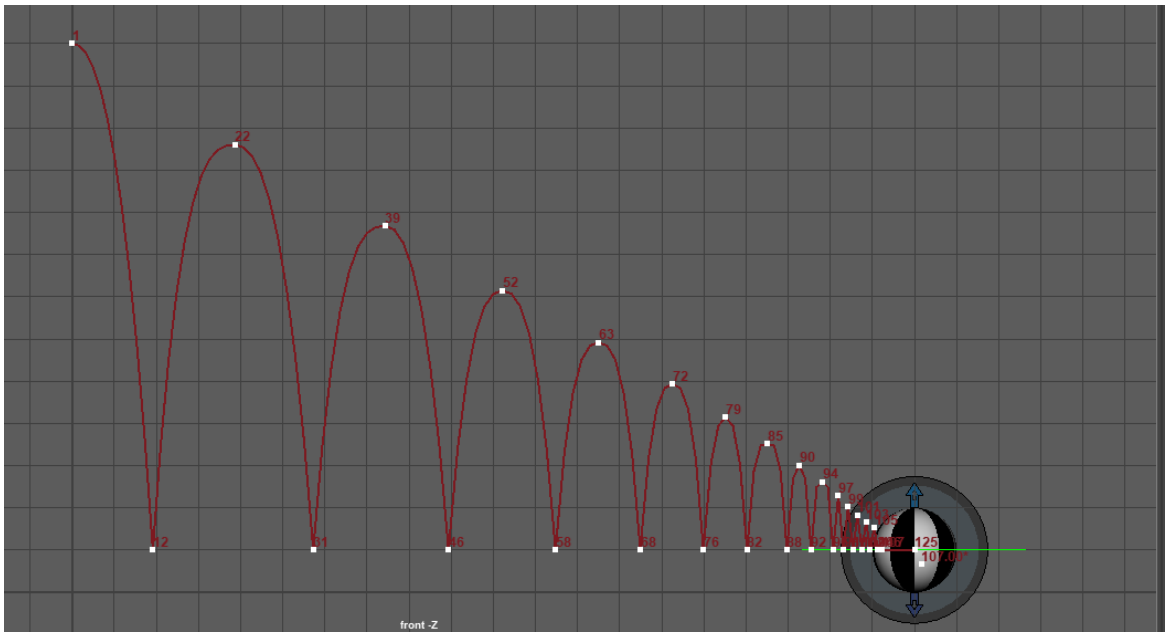
- Once we are happy with the translation of the ball along the Y and X-axis, as well as rotations we can start thinking of squash and stretch.
- The figure shows a frame from the animation sequence as well the a motion trail of the ball.



Combined Movement

Movement Along X/Y-axis & Rotation

- Once we are happy with the translation of the ball along the Y and X-axis, as well as rotations we can start thinking of squash and stretch.
- The figure shows a frame from the animation sequence as well the a motion trail of the ball.



References

- *PyMel* Lume Pictures GitHub repository
- *PyMel* “iamsleepy” GitHub repository.
- *PyMel* documentation.
- Timing and Spacing Video explanation
- “Bouncing Ball on-line Reference”